

Project Blueprint: Cloud-Based Plant Health AI Assistant

Dual-Model Cloud Multimodal AI with Go Proxy on Google Cloud Run (Two-Step Hybrid Pipeline)

Target Platform:	Android & iOS (React Native)
Target Market:	Philippines (Backyard gardeners, small-scale farmers, urban plant enthusiasts)
Operational Mode:	Online-First with Offline Knowledge Base Fallback
AI Models:	 Flash (Gemini 3.1 Flash Lite)  Deep Thinking (Gemma 4 31B)

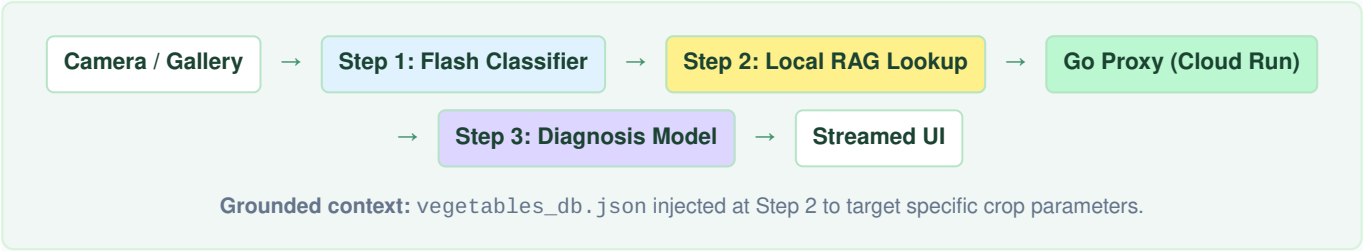
1. Overall System Pipeline

By replacing the locally trained vision model and local SLM with a cloud-based Gemini API, the pipeline is simplified. To avoid the high token cost of injecting the entire 143KB crop database (~35,000 tokens) on every request, the app uses a **Two-Step Hybrid Pipeline (Cloud Classifier + Local RAG)**.

Execution Flow

- Model Selection:** Before scanning, the user chooses between  **Flash** or  **Deep Thinking** mode.
- Image Capture:** The user takes a photo of a plant leaf using the React Native camera or selects a photo from the gallery. The image is compressed to 720p/1080p JPEG locally to save bandwidth.
- Step 1: Rapid Crop Classification (Router Call):** The app sends the compressed image to gemini-3.1-flash-lite via the Go Proxy. The prompt is a simple classification request containing just the list of 59 supported crop names (e.g., "What crop is in this image? Choose only from: Talong, Kamatis..."). This uses only ~350 tokens and returns the crop name in < 1 second.
- Step 2: Local RAG Context Injection:** The app looks up the identified crop name (e.g., "Talong") in the local vegetables_db.json and extracts only that crop's metadata section (~800 tokens).
- Step 3: Grounded Cloud AI Diagnosis:** The app sends the image + the single crop's metadata to the selected model ( **Flash** or  **Deep Thinking**) via the Go Proxy.
 -  **Flash** → gemini-3.1-flash-lite — Returns a fast, concise diagnosis in 1-2 seconds.
 -  **Deep Thinking** → gemma-4-31b — Returns a deep, detailed analysis with rich reasoning in 8-15 seconds.
- Go Proxy Relaying:** The Go Proxy (running on Google Cloud Run) securely attaches the Gemini API Key, relays the request, and handles the streamed response.
- Streamed Response UX:** The proxy streams the response back to the React Native app, displaying words live as they are generated.
- Follow-Up Chat:** The user can ask follow-up questions in a chatbot UI. The app maintains conversation context and applies the same model toggle.

Pipeline Diagram



2. Core System Components & Hardware Mapping

Component	Engine / Framework	Model / Service	Hardware Asset	Responsibility
Image Capture	React Native Camera + Image Picker	Native device camera or gallery	Device Camera Sensor / Storage	Captures the plant photo and compresses it locally to 720p/1080p JPEG.
Step 1 Classifier	Cloud Router via Go Proxy	gemini-3.1-flash-lite	Cloud (Google servers)	Automatically identifies the crop from the 59 supported crops in under 1 second (~350 tokens).
⚡ Flash Mode	Go Proxy + Gemini API	gemini-3.1-flash-lite	Cloud (Google servers)	Fast, lightweight diagnosis. Injects only the local RAG crop data. Returns concise response in 1-2 seconds.
🧠 Deep Thinking Mode	Go Proxy + Gemini API	gemma-4-31b	Cloud (Google servers)	Thorough diagnosis with deep reasoning. Injects only the local RAG crop data. Returns response in 8-15 seconds.
Secure Proxy Server	Go (Golang) Web Server	Custom Go Service on Google Cloud Run	Cloud (Google Run - Free Tier)	Secures the Gemini API Key. Manages the connection, sends requests, and handles raw HTTP stream relay (SSE) to client with zero timeout limits.
Follow-Up Chatbot	Go Proxy + Gemini API	User's selected model (⚡ or 🧠)	Cloud (Google servers)	Handles follow-up questions. Users can switch models mid-chat.
Knowledge Base	Local JavaScript Map / JSON	Bundled vegetables_db.json (< 1MB)	Device Storage / RAM	Stores grounded crop facts. Injects only the single identified crop metadata into Step 3 to save 96% token cost.
Offline Fallback	Local JSON Lookup	vegetables_db.json	Device Storage	When offline, disables AI and allows user to search the local database directly for symptoms/treatments.

What's Eliminated

The following components from the original offline blueprint are **no longer needed**:

Removed Component	Original Role	Why It's Removed
react-native-fast-tflite	On-device vision classification	Gemini API handles vision natively
Quantized .tflite model (~20–50MB)	Plant disease image classifier	No local model needed
llama.rn / react-native-llama	On-device text SLM	Gemini API handles text generation
GGUF model files (150MB–2GB)	Local language model weights	No local LLM needed
train.py pipeline	Model training + INT8 quantization	No training step required
download_images.py pipeline	Training dataset collection	No training dataset needed
"Choose Your Brain" tiered download	Hardware-adaptive model selection	Replaced by user-selectable ⚡ Flash / 🧠 Deep Thinking toggle

3. Local Knowledge Base Schema (vegetables_db.json)

The existing vegetables_db.json database is **fully retained** as-is. It serves as the grounded context injected into each Gemini API call. The total entries include 59 crops (30 vegetables/spices + 29 fruits), each with 2–3 health statuses (Healthy + 1–2 diseases).

```
{
  "Plant_Local_Name": {
    "local_name": "String (Filipino name)",
    "scientific_name": "String (Botanical name)",
    "category": "String (e.g., Vegetable, Fruit Tree, Spice)",
    "ecological_status": "String (e.g., Cultivated, Native, Naturalized)",
    "watering_needs": {
      "frequency": "String (Drought resistance & frequency guidelines)",
      "description": "String (Localized moisture advice)"
    },
    "description": "String (Cleaned general botanical description)",
    "habitat_ecology": "String (Regional climate details)",
    "uses": "String (Filipino culinary / medicinal / traditional use profile)",
    "health_statuses": {
      "Plant_Local_Name__Condition": {
        "condition_name": "String (Bilingual condition name)",
        "symptoms": "String (Visible identification marks)",
        "severity": "String (None / Low / Moderate / High)",
        "organic_treatment": "String (Natural/biological recipes)",
        "preventive_measures": "String (Cultural practices to avoid recurrence)"
      }
    }
  }
}
```

4. Dual-Model "Choose Your AI" System

4.1 Model Profiles

Feature	 Flash	 Deep Thinking
Model	gemini-3.1-flash-lite	gemma-4-31b
Speed	Very fast (1–3 seconds)	Slower (5–15 seconds)
Response Style	Concise bullet points, direct answers	Detailed paragraphs, thorough explanations
Best For	Quick field scans, fast checks while farming	Learning about diseases, in-depth understanding
Free Tier RPM	15 requests/min	15 requests/min
Free Tier TPM	250K tokens/min	Unlimited
Free Tier RPD	500 requests/day	1,500 requests/day
Vision / Text Chat	☒ Yes / ☒ Yes	☒ Yes / ☒ Yes

4.2 Where the Toggle Appears

- **Scan Screen (Before Capture):** A segmented control or pill toggle sits above the camera viewfinder / gallery upload button. The user selects their preferred mode before taking or uploading a photo.
- **Chat Screen (During Conversation):** After receiving an initial diagnosis, the user can continue asking follow-up questions. The same toggle appears in the chat input bar and can be switched mid-conversation.

4.3 API Configuration

Setting	Value
Client-Side SDK	Standard Fetch API (calling the Go Proxy server with streaming support)
Backend Go SDK	github.com/google/generative-ai-go/genai (Official Go SDK)
Deployment Platform	Google Cloud Run (Free Tier - 2 million requests/month)
Step 1 Model (Classifier)	gemini-3.1-flash-lite (~350 tokens)
Step 3 Model ( Flash)	gemini-3.1-flash-lite (~1,100 tokens with targeted context)
Step 3 Model ( Deep)	gemma-4-31b (~1,100 tokens with targeted context)
Output Format	Server-Sent Events (SSE) Streamed Text
Combined Free Tier	Up to 2,000 RPD total (500 Flash + 1,500 Deep Thinking)

4.4 Step 1: Crop Classification Prompt Template

You are a high-speed agricultural routing classifier. The user has uploaded an image of a crop leaf.

Identify which crop is in the image. You must choose ONLY from the following list of 59 supported crops:

[Talong, Kamatis, Sili, Sibuyas, Bawang, Luya, Gabi, Kamote, Kalabasa, ... (list of 59 crop names)]

Respond with ONLY the exact name of the crop from the list (e.g., "Talong"). Do not add any punctuation, explanation, or extra text.

4.5 Step 3: Grounded Diagnosis Prompt Template

You are a Filipino agricultural plant health assistant. The user has uploaded a photo of a {identified_crop} leaf.

Analyze the image and diagnose its health condition.

You must ground your response ONLY in the following verified metadata for {identified_crop}: {injected_crop_json_context}

Respond in this format:

- **Crop Identified:** [Local Name] ([Scientific Name])
- **Condition:** [Condition Name – bilingual]
- **Severity:** [Level]
- **Health Score:** [Provide a dynamic estimated health percentage from 0% to 100% based on visual leaf decay and diagnostic severity]%
- **Confidence Score:** [Provide a dynamic confidence score from 0% to 100% indicating how certain you are of this diagnosis based on image clarity, leaf symptom visibility, and similarity to database descriptions]%
- **Symptoms Observed:** [What you see in the image matching the database]
- **Organic Treatment:** [From database]
- **Prevention:** [From database]
- **Care Tip:** [A friendly, localized tip]

Keep your language warm, supportive, and accessible to Filipino farmers.
Mix English and Filipino naturally (Taglish) when appropriate.

4.6 System Prompt Template (Chat / Follow-Up Mode)

You are a friendly Filipino plant care chatbot. The user previously scanned a plant and received the following diagnosis:
{previous_diagnosis_result}

The user may ask follow-up questions about treatment steps, disease prevention, watering schedules, or general care for this crop.
Use ONLY the following verified crop database as your reference:
{injected_crop_json_context}

Rules:

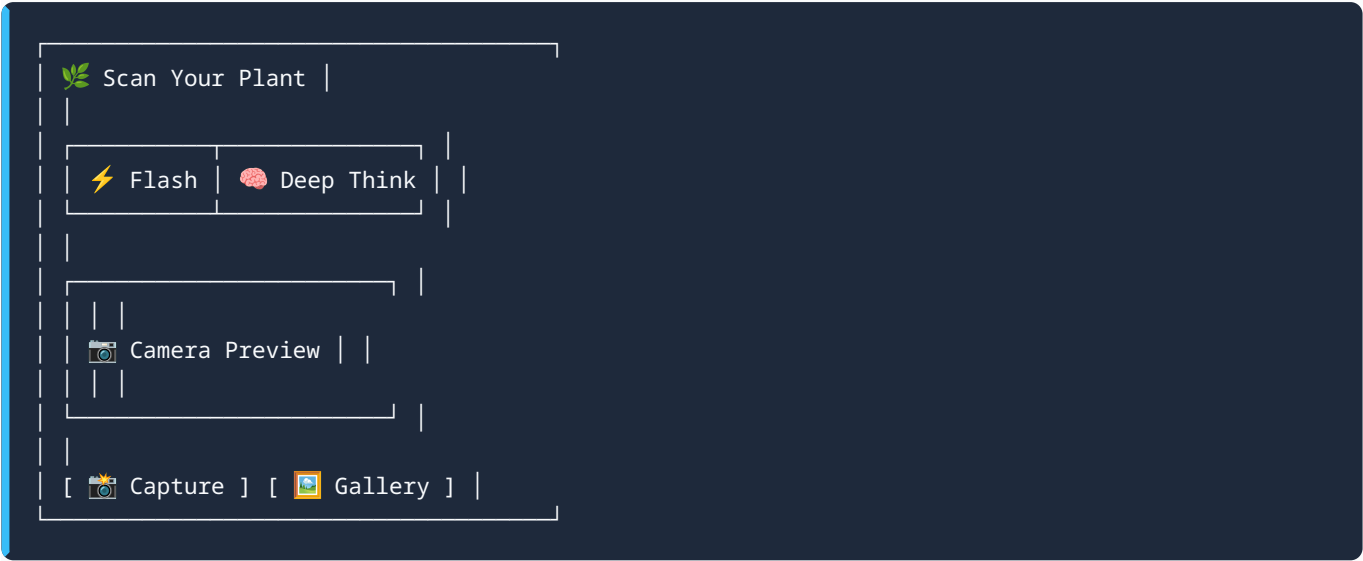
- Stay within the scope of the 59 crops in the database.
- If the user asks about a crop or topic not in the database, politely say you can only help with the supported Philippine crops.
- Use warm, encouraging Taglish when appropriate.
- Provide actionable, practical advice suitable for backyard farming.

4.7 Cost Estimation (Free Tier — Both Models Combined)

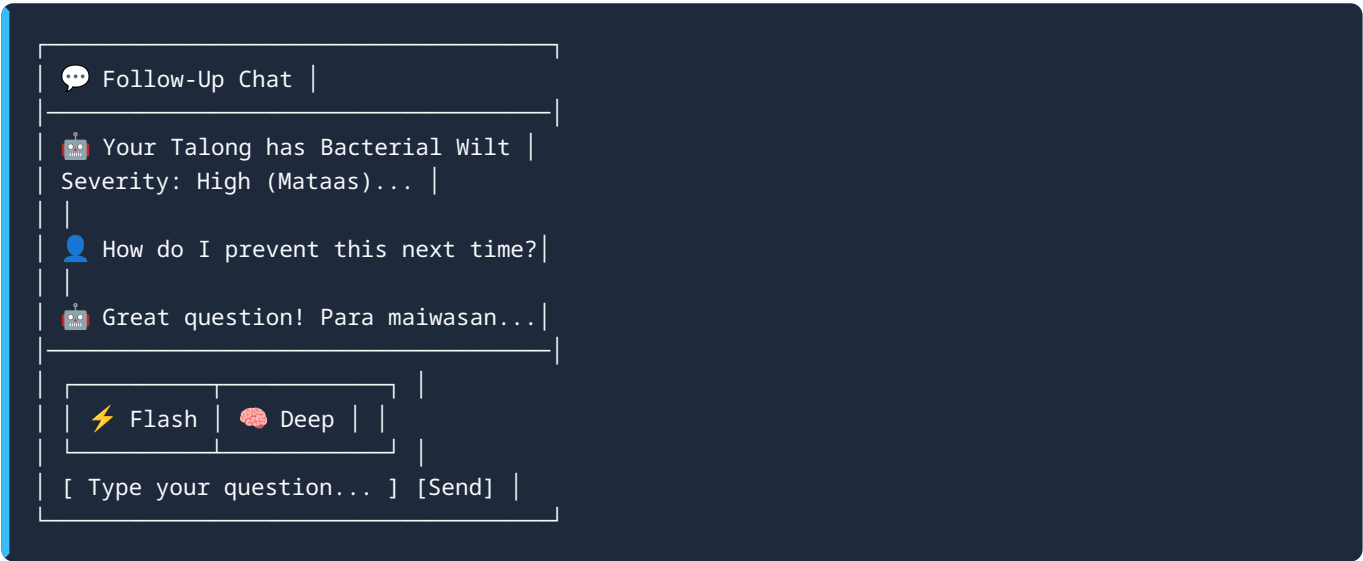
Usage Pattern	⚡ Flash Usage	🧠 Deep Usage	Total RPD	Cost
Development/Testing	30 scans	20 scans	50	Free
Soft Launch (100 users)	150 scans	50 deep scans	200	Free
Growth (500 users)	400 scans + chats	200 deep scans	600	Free (split across models)
Production (1000+ users)	500+	500+	1000+	May need paid tier

5. UX Flow: "Choose Your AI" Toggle & Bento Layout

5.1 Scan Screen Layout



5.2 Chat Screen Layout



5.3 Scan Results Dashboard (Bento Grid & Circle Progress)

Once the AI diagnosis returns, the information is structured into a modern **Bento Grid Layout** to make complex details easy to read at a glance.

5.3.1 Health Indicator (Circle Progress Bar)

A prominent indicator visualizes the plant's health score. The score color matches its diagnostic severity:

-  **70% - 100% (Green): Excellent / Healthy** — Injects care tips and standard preventive practices.
-  **40% - 69% (Orange/Yellow): Good / Mild Damage** — Injects immediate organic treatment.
-  **0% - 30% (Red): Critical / Severe Disease** — Requires immediate quarantine or eradication.

5.3.2 Bento Grid Layout Schema (React Native Flexbox)

TILE 1: HERO (2X2 SPAN)

Talong

Solanum melongena

[Crop Photo / Thumbnail Placeholder]

TILE 2: HEALTH STATUS

15%

Red / Critical

TILE 5: ECOLOGICAL CAT

Category: Vegetable

Ecological: Cultivated

TILE 3: CONDITION, SEVERITY & CONFIDENCE (2X1 SPAN)

Condition: Bacterial Wilt (Layong Baktery)

Severity: High (Mataas) Confidence: 95%

TILE 4: WATERING NEEDS

Freq: Low (2-3 days)

Avoid soggy soil.

CONTEXT TARGET

Grounded to local database.

TILE 6: OBSERVED SYMPTOMS (2X1 SPAN)

Sudden wilting of leaves during sunny days

Stems showing dark vascular discoloration

TILE 7: STEPS TO TREAT (ORGANIC RECIPE) (2X2 SPAN)

1. Pull out the infected plant immediately and burn.

2. Treat soil with copper-based organic fungicides.

TILE 8: STEPS TO PREVENT (2X1 SPAN)

Practice 3-year crop rotation with non-solanaceous.

Use disease-free certified seeds.

5.4 Model Behavior Differences in Practice

Example: User scans a Talong leaf with Bacterial Wilt

⚡ Flash Response (~2 seconds):

Crop: Talong (Solanum melongena) | **Condition:** Bacterial Wilt (Layong Baktery) | **Severity:** High | **Health Score:** 15% | **Confidence Score:** 92%
Treatment: Uproot and burn immediately. Practice crop rotation.
Tip: Avoid planting tomatoes or peppers in the same spot next season.

🧠 Deep Thinking Response (~8 seconds):

Crop Identified: Talong — Solanum melongena (Eggplant)
Condition: Bacterial Wilt (Layong Baktery) — caused by Ralstonia solanacearum
Based on the image, I can see rapid wilting of the upper leaves despite what appears to be moist soil

Page 8 of 13

at the base. This is a classic hallmark of bacterial wilt...

Severity: High (Mataas) — Kailangan agad-agad na aksyon. | **Health Score:** 15% | **Confidence Score:** 95%

Step-by-Step Organic Treatment: 1. Bunutin agad ang buong halaman... 2. Sunugin o ilubog nang malalim...

6. Adaptive Connectivity Workflow

Connectivity State	System Behavior
Online	Full AI-powered diagnosis using the user's selected model (⚡ or 🧠). Image analysis + streamed conversational response + follow-up chat.
Offline	Model toggle is disabled (grayed out). User manually selects their crop and condition from a searchable local list. App displays the raw vegetables_db.json data (symptoms, treatment, severity) in a clean card layout.
Slow/Unstable Connection	App attempts the AI call with a 10-second timeout. If it fails, auto-switches to offline mode with a toast: "Walang internet — nagpapakita ng local na impormasyon."

7. Critical Engineering Rules for Development

- API Key Security (Go Proxy):** Never embed the Gemini API key directly in the React Native JavaScript bundle. Deploy a lightweight backend proxy written in **Go (Golang)** to Google Cloud Run to securely attach the key and relay requests.
- Two-Step Grounding (96% Token Savings):** Do not send the entire vegetables_db.json database. Use the 2-step hybrid approach: Call gemini-3.1-flash-lite with a minimal prompt containing just the 59 crop names to identify the crop class (~350 tokens). Then retrieve that specific crop's metadata from vegetables_db.json locally and inject only that section into the final diagnostic call (~1,100 tokens total).
- Image Optimization:** Before sending to the API, compress and resize the camera image to ~720p JPEG quality to minimize upload time and token consumption on mobile data connections.
- Streaming UX (Google Cloud Run):** Stream Gemini's response chunk-by-chunk using Server-Sent Events (SSE). Hosting on Google Cloud Run ensures the connection remains open for long detailed responses without the 10-second serverless timeouts seen on Vercel. Combine with progressive loading states to manage expectations during deep thinking modes.
- Offline-First Data:** Bundle vegetables_db.json directly in the app assets. The app must remain useful even without internet — the local database alone provides all symptoms, treatments, and prevention data.
- Rate Limit Handling:** Track usage per model separately. If ⚡ Flash hits its 500 RPD limit, suggest the user switch to 🧠 Deep Thinking (which has 1,500 RPD).
- Chat History Management:** Maintain conversation context per session. When the user switches models mid-chat, pass the existing conversation history to the new model so context is preserved. Clear chat history when a new scan is started.
- Model Toggle State Persistence:** Save the user's preferred model choice to AsyncStorage so it persists across app restarts. Default to ⚡ Flash for first-time users.

8. Database Architecture (Supabase + SQLite via TanStack Query)

To support secure user accounts, cloud image hosting, and robust offline-first functionality, the app uses a **Hybrid Database System** pairing **Supabase (Cloud)** with **Expo SQLite + TanStack Query (Local Client)**.

8.1 Technical Roles & Data Flow

- **Supabase Auth:** Handles user authentication (login, signup, session persistence).
- **Supabase Storage:** Hosts uploaded plant leaf images under a `plant-images` bucket.
- **Supabase PostgreSQL:** Stores authoritative user profiles, scan histories, and chat logs.
- **Expo SQLite:** Local database on the device. Syncs with Supabase when online. Acts as the instant-load query cache and offline registry.
- **TanStack Query (React Query):** Orchestrates synchronization. Handles offline query caching, request retries, and offline mutation queuing.

8.2 Database Schema (Mirrored Local & Cloud)

Table: scans

Column	Type	Target	Description
id	UUID	Both	Primary Key. Generated on client.
user_id	UUID	Both	Foreign Key to Supabase Auth (<code>auth.uid()</code>).
crop_name	Text	Both	E.g., "Talong".
condition_name	Text	Both	E.g., "Bacterial Wilt (Layong Bakteryang)".
severity	Text	Both	"None", "Low", "Moderate", "High".
health_score	Integer	Both	Dynamic AI health percentage (0-100).
confidence_score	Integer	Both	Dynamic AI confidence percentage (0-100).
local_image_path	Text	SQLite Only	Local URI of the file on device.
cloud_image_url	Text	Both	Public URL of the image stored in Supabase Storage.
diagnosis_text	Text	Both	Markdown response returned by the Gemini/Gemma models.
synced	Boolean	SQLite Only	<code>true</code> if uploaded to Supabase, <code>false</code> if pending sync.
created_at	Timestamp	Both	When the scan was taken. Default: <code>now()</code> .

Table: profiles

Column	Type	Target	Description
id	UUID	Both	Primary Key. References auth.users(id) with cascade delete.
username	Text	Both	Unique username.
full_name	Text	Both	Display name.
avatar_url	Text	Both	Public URL of the profile picture in Supabase Storage.
updated_at	Timestamp	Both	When the profile was last modified.

Table: chat_sessions

Column	Type	Target	Description
id	UUID	Both	Primary Key. Generated on client.
scan_id	UUID	Both	Foreign Key to scans.id.
user_id	UUID	Both	Foreign Key to Supabase Auth.
title	Text	Both	E.g., "Follow-up on Talong".
synced	Boolean	SQLite Only	Sync status.
created_at	Timestamp	Both	When the session was started.

Table: chat_messages

Column	Type	Target	Description
id	UUID	Both	Primary Key. Generated on client.
session_id	UUID	Both	Foreign Key to chat_sessions.id.
sender	Text	Both	"user" or "ai".
model_used	Text	Both	"gemini-3.1-flash-lite" or "gemma-4-31b".
message	Text	Both	Text body of the message.
synced	Boolean	SQLite Only	Sync status.
created_at	Timestamp	Both	Message timestamp.

8.3 Offline Sync Logic

- Write Local:** When a user initiates a scan or types a chat message, the app generates a UUIDv4 and instantly inserts the record into SQLite (with `synced = false`).
- Network Detection:** The app listens to connection events using `@react-native-community/netinfo`.

3. **Background Sync Queue:** When online, TanStack Query processes the mutation queue. For a scan: the app uploads the local image file to Supabase Storage, receives the `cloud_image_url`, updates Supabase Postgres, and then sets `synced = true` locally.

8.4 User Authentication & Registration Flow

1. **Initial Development (Classic Email/Password Auth):** User signs up via `supabase.auth.signUp()`. A PostgreSQL trigger automatically provisions a matching row in `public.profiles`.
2. **Offline Registration Resilience:** Authentication requires active internet. Once authenticated, the profile is cached locally in SQLite for offline access.
3. **Future OAuth Extensibility:** Designed to scale cleanly with Google & Facebook logins via the same database trigger mechanisms.

9. Migration Checklist (From Offline to Cloud Architecture)

API & Backend

- ☐ Set up a Google AI Studio project and obtain a Gemini API key
- ☐ Initialize the Go proxy codebase (`go mod init`, install official Go GenAI SDK)
- ☐ Implement Go HTTP server with streaming support (Server-Sent Events) and security proxy logic
- ☐ Create a Dockerfile for the Go proxy server and deploy to Google Cloud Run
- ☐ Configure environment variables in Google Cloud Run to hold the `GEMINI_API_KEY` securely
- ☐ Create a Supabase project and set up the `profiles`, `scans`, `chat_sessions`, and `chat_messages` tables
- ☐ Set up a PostgreSQL trigger function to automatically create a `profiles` row when a user registers
- ☐ Set up Row-Level Security (RLS) policies on Supabase to ensure users can only access their own data
- ☐ Create Supabase Storage buckets: `plant-images` and `avatars` with public read access

Core Features

- ☐ Build the ⚡ / 🧠 model toggle component (segmented control / pill switch)
- ☐ Implement the camera capture + gallery upload flow with image compression
- ☐ Implement **Step 1 (Crop Classifier)** API call to identify the crop class
- ☐ Implement local JSON extraction of the identified crop metadata
- ☐ Implement **Step 3 (Diagnostic Model)** API call with targeted context and streaming response
- ☐ Build the follow-up chatbot screen with model toggle and chat history management
- ☐ Save model preference to `AsyncStorage`
- ☐ Install `@supabase/supabase-js`, `expo-sqlite`, and `@tanstack/react-query` in the React Native project
- ☐ Set up the local SQLite database schema on app startup
- ☐ Build User Registration, Login, and profile picture upload screens using the Supabase client SDK

Offline & Connectivity

- ☐ Add connectivity detection (`@react-native-community/netinfo`)
- ☐ Build the offline fallback UI (searchable crop list + raw data cards)
- ☐ Disable model toggle and show offline indicator when no internet is available
- ☐ Implement background synchronization worker to upload pending SQLite records (`synced = false`)

Cleanup & Testing

- ☐ Remove `react-native-fast-tflite`, `llama.rn`, and all local model weights
- ☐ Remove `train.py` and `download_images.py` from the project codebase
- ☐ Test on low-spec budget devices to verify reduced memory and storage footprints
- ☐ Test both models with real plant photos via the Go proxy on Cloud Run
- ☐ Verify offline fallback behavior, image uploads, and auto-synchronization when connection changes
- ☐ Monitor API usage against free tier limits for both models